

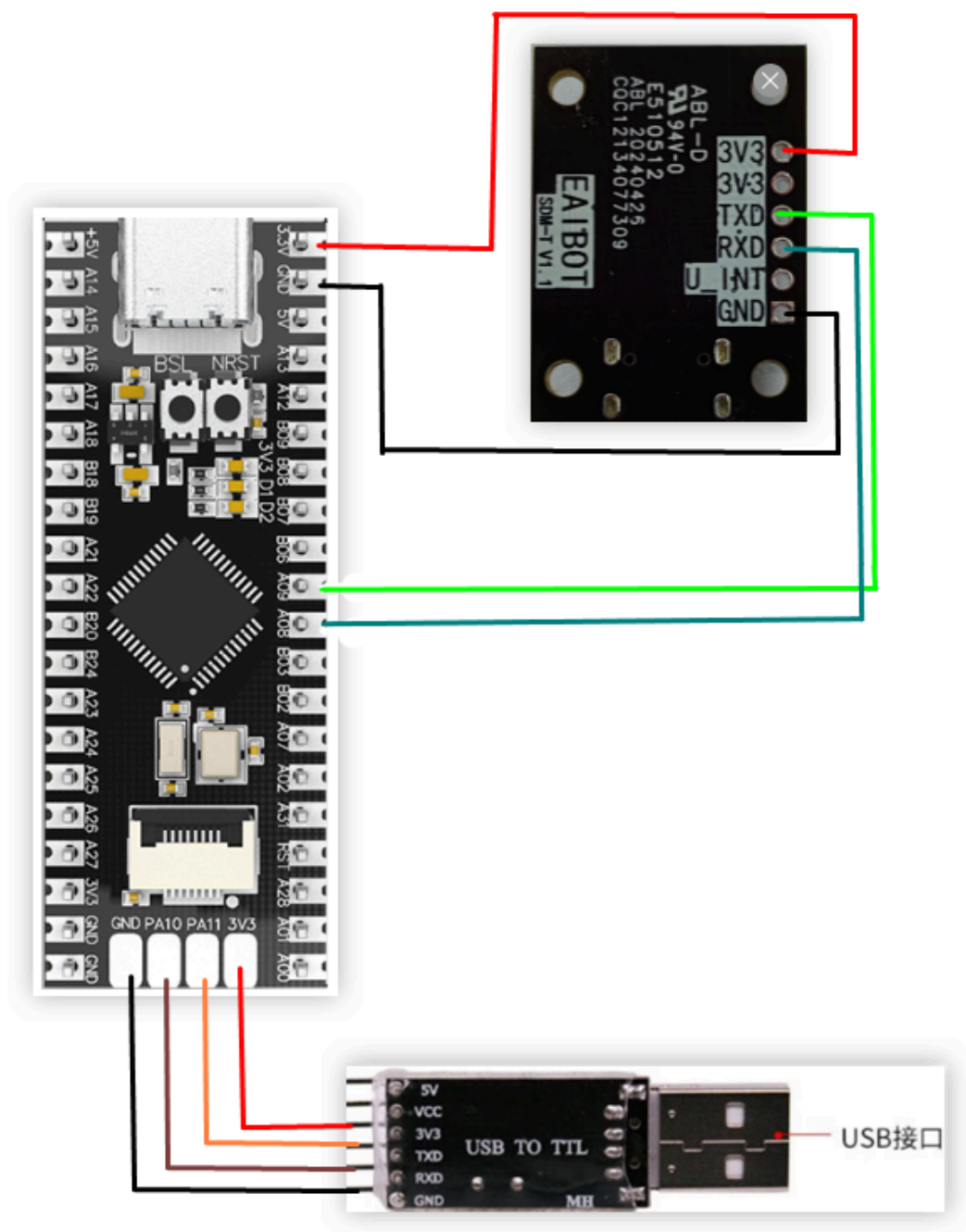
单点激光测距

一、学习目标

通过单点激光测距模块测距并通过串口打印到电脑的串口助手。

二、硬件连接

单点激光测距模块、USB转TTL与MSPM0G3507接线



注：如果没有TTL模块的，也可以直接用type-c串口

三、SDM18的一些重要参数讲解

1. SDM18接口如图所示:

SDM18 对外物理接口端子为 WF08006，实现系统供电和数据通信功能。

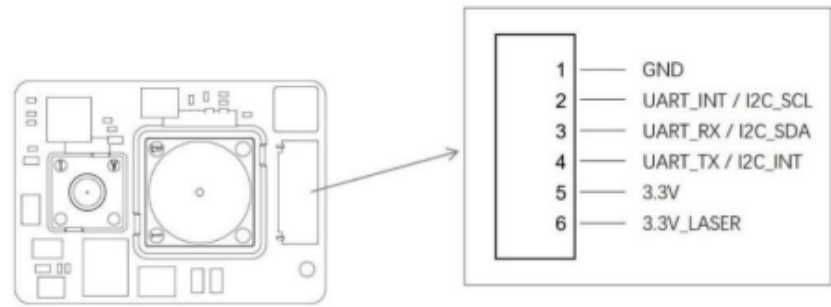


图 2 YDLIDAR SDM18 物理接口

2. 串口默认配置

SDM18默认的波特率为921600

接口	最小值	典型值	最大值	单位	备注
UART	9600	921600	921600	bps	信号电平 3.3V， 8 位数据位，1 位停止位， 无校验

3. 基于串口的重要协议命令(都是16进制的形式发送)

SDM18上电的默认模式是空闲模式，如果要进行测距要先开始测距的命令

- 开始测距: A5 03 20 01 00 00 00 02 6E
- 停止测距: A5 03 20 02 00 00 00 00 46 6E

接收测距的协议解析

包头	设备号	设备类型	命令类型	预留位	数据长度	数据段	校验码
1 Byte	1 Byte	1 Byte	1 Byte	1 Byte	2 Bytes	N Bytes	2 Bytes

- **包头**：SDM18 的报文包头标志为 0xA5；
- **设备号**：SDM18 的报文设备号标志为 0x03；
- **设备类型**：根据下位机评估板的类型而定，为 0x20；
- **命令类型**：系统命令码，见表 1；
- **预留位**：预留状态位，以留后续使用；
- **数据长度**：表示的是应答数据的长度；
- **数据段**：不同系统命令下的应答内容，反馈不同的数据内容，其数据格式也不同；
- **校验码**：除去校验码以外，所有数据的 CRC16 校验结果。

包头	设备号	设备类型	命令类型	预留位	数据长度	数据段	校验码
0xA5	0x03	0x20	0x01	0x00	0x00 0E		CRC16

设应答内容为：A5 03 20 01 00 00 0E FF FF FF FF FF F6 06 42 00 74 26 01 00 0B 44，数据长度 = 0x00 0E，即数据段字节数为 14；则数据段为 FF FF FF FF FF FF F6 06 42 00 74 26 01 00，其内容满足以下数据结构：

字节偏移	0	1	2	3	4	5	6	7	8	9	10	11	12	13

距离值 强度值 预留位

F6 06: 距离值为 0x06F6 = 1782mm

74 26: 强度值为 0x2674 = 9844

4. 校验码的计算

该校验码是才用了CRC-16-modelbus的计算方式，把除了校验位以外的字节都进行计算

下图是这**开始测距**这条命令的校验位计算结果

输入内容

A5 03 20 01 00 00 00

内容格式

Hex

算法选择

CRC-16-MODBU

计算

清空

多项式公式

x16+x15+x2+1

宽度位数

16

多项式POLY(HEX)

8005

初始值INIT(HEX)

FFFF

结果异或值XOROUT(HEX)

0000

数据反转

输入反转(REFIN) ☒

输出反转(REFOUT) ☒

计算结果(HEX)

026E

计算结果(HEX)

00000010 04104440

其它协议不再讲解，有兴趣的去看SDM18开发手册

四、程序说明

- usart.c

```
void USART_Init(void)
{
    // SYSCFG初始化
```

```

// SYSCFG initialization
SYSCFG_DL_init();
//清除串口中断标志
//Clear the serial port interrupt flag
NVIC_ClearPendingIRQ(UART_0_INST_INT_IRQN);
//使能串口中断
//Enable serial port interrupt
NVIC_EnableIRQ(UART_0_INST_INT_IRQN);

NVIC_ClearPendingIRQ(UART_1_INST_INT_IRQN);
NVIC_EnableIRQ(UART_1_INST_INT_IRQN);

}

//USART1发送一个字符
//USART1 sends a character
void USART1_Send_U8(uint8_t data)
{
    //当串口1忙的时候等待
    //Wait when serial port 1 is busy
    while( DL_UART_isBusy(UART_1_INST) == true );
    //发送 Send
    DL_UART_Main_transmitData(UART_1_INST, data);;
}

//串口1发送N个字符
//Serial port 1 sends N characters
void USART1_Send_ArrayU8(uint8_t *pData, uint16_t Length)
{
    while (Length--)
    {
        USART1_Send_U8(*pData);
        pData++;
    }
}

//串口的中断服务函数
//Serial port interrupt service function
void UART_0_INST_IRQHandler(void)
{
    uint8_t receivedData = 0;

    //如果产生了串口中断
    //If a serial port interrupt occurs
    switch( DL_UART_getPendingInterrupt(UART_0_INST) )
    {
        case DL_UART_IIDX_RX://如果是接收中断 If it is a receive interrupt

            // 接收发送过来的数据保存 Receive and save the data sent
            receivedData = DL_UART_Main_receiveData(UART_0_INST);

            if((recv0_length&0x8000)==0)//接收未完成 Receiving is not completed
            {
                if(recv0_length&0x4000)//接收到了0x0d Received 0x0d
                {

```

```

        if(receivedData!=0x0a)recv0_length=0;//接收错误,重新开始    Receive
error, restart
        else recv0_length|=0x8000;  //接收完成了 Receiving completed
    }
    else //还没收到0X0D Haven't received 0X0D yet
    {
        if(receivedData==0x0d)recv0_length|=0x4000;
        else
        {
            recv0_buff[recv0_length&0X3FFF]=receivedData ;
            recv0_length++;
            if(recv0_length>(RE_0_BUFF_LEN_MAX-1))recv0_length=0;//接收数
据错误,重新开始接收    Receive data error, restart receiving
        }
    }
}

break;

default://其他的串口中断    Other serial port interrupts
break;
}
}

//串口1的中断服务函数
//Interrupt service function of serial port 1
void UART_1_INST_IRQHandler(void)
{
    uint8_t receivedData = 0;

    //如果产生了串口中断
    //If a serial port interrupt occurs
    switch( DL_UART_getPendingInterrupt(UART_1_INST) )
    {
        case DL_UART_IIDX_RX://如果是接收中断    If it is a receive interrupt

            // 接收发送过来的数据保存
            //Receive and save the data sent
            receivedData = DL_UART_Main_receiveData(UART_1_INST);
            SDM18_Decode(receivedData);
            break;

            default://其他的串口中断    Other serial port interrupts
            break;
        }
    }
}

```

- USART_Init: 串口0与串口1的初始化函数。
- USART1_Send_U8、USART1_Send_ArrayU8: 串口1的发送函数, 用于向SDM18发送指令。
- UART_0_INST_IRQHandler: 串口0的接收中断, 接收到数据后可以将其打印在串口助手上。
- UART_1_INST_IRQHandler: 串口1的接收中断, 用于接收SDM18发送过来的数据, 并检验。

- empty.c

```
int main(void)
{
    USART_Init();
    printf("waiting for SDM18 start work!\r\n");
    SMD18_init(B921600); //SDM18初始化  SDM18 Initialization
    while(1)
    {
        //stop_scan();
        if(newlines == 1)
        {
            newlines = 0; //清掉它  Clear it
            //打印信息  Print information
            print_message();
            delay_ms(50);
        }
    }
}
```

初始化串口和SDM18，并每隔50毫秒在串口打印SDM18传输回来的测量数据。

注：必须将工程源码放在SDK路径下进行编译，

例如路径：D:\TI\M0_SDK\mspm0_sdk_1_30_00_03\1.TB6612

新加卷 (D:) > TI > M0_SDK > mspm0_sdk_1_30_00_03				
名称	修改日期	类型	大小	
1.TB6612	2024/7/22 18:59	文件夹		
2.AT8236	2024/7/22 19:47	文件夹		
3.Enconder	2024/7/23 10:36	文件夹		
4.Servo	2024/7/23 11:13	文件夹		
docs	2024/7/23 10:33	文件夹		
examples	2024/7/23 10:34	文件夹		
kernel	2024/7/23 10:37	文件夹		
source	2024/7/23 10:33	文件夹		
tools	2024/7/23 10:33	文件夹		
imports.mak	2024/1/25 11:45	MAK 文件	2 KB	
known_issues_FAQ.html	2024/1/25 11:42	Microsoft Edge ...	67 KB	
license_mspm0_sdk_1_30_00_03.txt	2024/1/25 11:42	文本文档	33 KB	
manifest_mspm0_sdk_1_30_00_03.html	2024/1/25 11:42	Microsoft Edge ...	113 KB	
mspm0sdk_1_30_00_03.log	2024/7/23 10:42	文本文档	5,237 KB	
release_notes_mspm0_sdk_1_30_00_0...	2024/1/25 11:42	Microsoft Edge ...	108 KB	
uninstall.dat	2024/7/23 10:39	DAT 文件	344 KB	
uninstall.exe	2024/7/23 10:39	应用程序	6,048 KB	

五、实验现象

给MSPM0G3507烧录程序，按照接线图接好线之后。关闭其他占用串口的程序，电脑打开串口助手，选择串口号，波特率设置成115200。在串口助手会看打印的距离数据，单位是毫米（mm）和强度信息

效果图：

```
[2024-12-14 12:39:53.754]# RECV ASCII>
dis = 194mm      , strength = 9125

[2024-12-14 12:39:54.323]# RECV ASCII>
dis = 188mm      , strength = 9880

[2024-12-14 12:39:54.894]# RECV ASCII>
dis = 102mm      , strength = 9845

[2024-12-14 12:39:55.466]# RECV ASCII>
dis = 142mm      , strength = 9873

[2024-12-14 12:39:56.037]# RECV ASCII>
dis = 167mm      , strength = 9880

[2024-12-14 12:39:56.606]# RECV ASCII>
dis = 133mm      , strength = 9880
```